

بسم الله الرحمن الرحيم

King Abdulaziz University  
Faculty of Computing and information Technology  
Computer Science Department



# CPCS-214 Computer Architecture and Organization

## 06 – Instructions: Language of Computer

# Introduction

- How to command computer?
  - To command computer 's hardware you must speak its language
- What language computer can understand?
- Words of a computer language: **instructions**
- Its vocabulary is called: **instruction set**

# Instruction Sets

- Different computers have different instruction sets
  - But with many aspects in common
- Early computers had very simple instruction sets
  - Simplified implementation
- Many modern computers also have simple instruction sets

# Instruction Sets

- You will see the instruction set of a real computer, both
  - in the form written by people
  - in the form read by the computer

# Instruction Sets

- Computer languages are quite similar, more like regional dialects than like independent languages
- Hence, once you learn one, it is easy to pick up others
- This similarity of instruction sets occurs because
  - All computers are constructed from HW based on similar principles
  - There are a few basic operations that all computers must provide

# Designers Goal

- To find a language that
  - makes it easy to build HW and compiler
  - maximizing performance
  - minimizing cost and energy

# MIPS

- Chosen instruction set comes from MIPS Technologies
- Elegant example of the instruction sets designed since 1980s

# MIPS Instruction Set

- Used throughout the course
- Stanford MIPS commercialized by MIPS Technologies
- Large share of embedded core market
  - Applications

# Stored-program Concept

- By learning how to represent instructions, you will also discover the secret of computing: **stored-program concept**
  - Idea that instructions and data of many types can be stored in memory as numbers

# MIPS Instruction Set

- Next figures gives a sneak preview of some of the instruction set to be covered

# MIPS Set

## MIPS operands

| Name                  | Example                                                                                       | Comments                                                                                                                                                                                                                |
|-----------------------|-----------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 32 registers          | <code>\$s0-\$s7, \$t0-\$t9, \$zero, \$a0-\$a3, \$v0-\$v1, \$gp, \$fp, \$sp, \$ra, \$at</code> | Fast locations for data. In MIPS, data must be in registers to perform arithmetic, register <code>\$zero</code> always equals 0, and register <code>\$at</code> is reserved by the assembler to handle large constants. |
| $2^{30}$ memory words | <code>Memory[0], Memory[4], . . . , Memory[4294967292]</code>                                 | Accessed only by data transfer instructions. MIPS uses byte addresses, so sequential word addresses differ by 4. Memory holds data structures, arrays, and spilled registers.                                           |

# MIPS Operands

| Name                  | Example                                                                                       | Comments                                                                                                                                                                                                                |
|-----------------------|-----------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 32 registers          | <code>\$s0-\$s7, \$t0-\$t9, \$zero, \$a0-\$a3, \$v0-\$v1, \$gp, \$fp, \$sp, \$ra, \$at</code> | Fast locations for data. In MIPS, data must be in registers to perform arithmetic, register <code>\$zero</code> always equals 0, and register <code>\$at</code> is reserved by the assembler to handle large constants. |
| $2^{30}$ memory words | <code>Memory[0], Memory[4], . . . , Memory[4294967292]</code>                                 | Accessed only by data transfer instructions. MIPS uses byte addresses, so sequential word addresses differ by 4. Memory holds data structures, arrays, and spilled registers.                                           |

# MIPS assembly language

| Category           | Instruction                      | Example             | Meaning                                                   | Comments                              |
|--------------------|----------------------------------|---------------------|-----------------------------------------------------------|---------------------------------------|
| Arithmetic         | add                              | add \$s1,\$s2,\$s3  | $\$s1 = \$s2 + \$s3$                                      | Three register operands               |
|                    | subtract                         | sub \$s1,\$s2,\$s3  | $\$s1 = \$s2 - \$s3$                                      | Three register operands               |
|                    | add immediate                    | addi \$s1,\$s2,20   | $\$s1 = \$s2 + 20$                                        | Used to add constants                 |
| Data transfer      | load word                        | lw \$s1,20(\$s2)    | $\$s1 = \text{Memory}[\$s2 + 20]$                         | Word from memory to register          |
|                    | store word                       | sw \$s1,20(\$s2)    | $\text{Memory}[\$s2 + 20] = \$s1$                         | Word from register to memory          |
|                    | load half                        | lh \$s1,20(\$s2)    | $\$s1 = \text{Memory}[\$s2 + 20]$                         | Halfword memory to register           |
|                    | load half unsigned               | lhu \$s1,20(\$s2)   | $\$s1 = \text{Memory}[\$s2 + 20]$                         | Halfword memory to register           |
|                    | store half                       | sh \$s1,20(\$s2)    | $\text{Memory}[\$s2 + 20] = \$s1$                         | Halfword register to memory           |
|                    | load byte                        | lb \$s1,20(\$s2)    | $\$s1 = \text{Memory}[\$s2 + 20]$                         | Byte from memory to register          |
|                    | load byte unsigned               | lbu \$s1,20(\$s2)   | $\$s1 = \text{Memory}[\$s2 + 20]$                         | Byte from memory to register          |
|                    | store byte                       | sb \$s1,20(\$s2)    | $\text{Memory}[\$s2 + 20] = \$s1$                         | Byte from register to memory          |
|                    | load linked word                 | ll \$s1,20(\$s2)    | $\$s1 = \text{Memory}[\$s2 + 20]$                         | Load word as 1st half of atomic swap  |
|                    | store condition. word            | sc \$s1,20(\$s2)    | $\text{Memory}[\$s2 + 20] = \$s1; \$s1 = 0 \text{ or } 1$ | Store word as 2nd half of atomic swap |
|                    | load upper immed.                | lui \$s1,20         | $\$s1 = 20 * 2^{16}$                                      | Loads constant in upper 16 bits       |
| Logical            | and                              | and \$s1,\$s2,\$s3  | $\$s1 = \$s2 \& \$s3$                                     | Three reg. operands; bit-by-bit AND   |
|                    | or                               | or \$s1,\$s2,\$s3   | $\$s1 = \$s2   \$s3$                                      | Three reg. operands; bit-by-bit OR    |
|                    | nor                              | nor \$s1,\$s2,\$s3  | $\$s1 = \sim (\$s2   \$s3)$                               | Three reg. operands; bit-by-bit NOR   |
|                    | and immediate                    | andi \$s1,\$s2,20   | $\$s1 = \$s2 \& 20$                                       | Bit-by-bit AND reg with constant      |
|                    | or immediate                     | ori \$s1,\$s2,20    | $\$s1 = \$s2   20$                                        | Bit-by-bit OR reg with constant       |
|                    | shift left logical               | sll \$s1,\$s2,10    | $\$s1 = \$s2 \ll 10$                                      | Shift left by constant                |
|                    | shift right logical              | srl \$s1,\$s2,10    | $\$s1 = \$s2 \gg 10$                                      | Shift right by constant               |
| Conditional branch | branch on equal                  | beq \$s1,\$s2,25    | if ( $\$s1 == \$s2$ ) go to PC + 4 + 100                  | Equal test; PC-relative branch        |
|                    | branch on not equal              | bne \$s1,\$s2,25    | if ( $\$s1 \neq \$s2$ ) go to PC + 4 + 100                | Not equal test; PC-relative           |
|                    | set on less than                 | slt \$s1,\$s2,\$s3  | if ( $\$s2 < \$s3$ ) $\$s1 = 1$ ; else $\$s1 = 0$         | Compare less than; for beq, bne       |
|                    | set on less than unsigned        | sltu \$s1,\$s2,\$s3 | if ( $\$s2 < \$s3$ ) $\$s1 = 1$ ; else $\$s1 = 0$         | Compare less than unsigned            |
|                    | set less than immediate          | slti \$s1,\$s2,20   | if ( $\$s2 < 20$ ) $\$s1 = 1$ ; else $\$s1 = 0$           | Compare less than constant            |
|                    | set less than immediate unsigned | sltiu \$s1,\$s2,20  | if ( $\$s2 < 20$ ) $\$s1 = 1$ ; else $\$s1 = 0$           | Compare less than constant unsigned   |
| Unconditional jump | jump                             | j 2500              | go to 10000                                               | Jump to target address                |
|                    | jump register                    | jr \$ra             | go to \$ra                                                | For switch, procedure return          |
|                    | jump and link                    | jal 2500            | $\$ra = \text{PC} + 4$ ; go to 10000                      | For procedure call                    |